

SHUNYA

Architecture Report

2026-07-14

Read-only forensic architecture inspection

Prepared for: Nishesh Singhal

SHUNYA ARCHITECTURE REPORT — 2026-07-14

1. REPOSITORY STATE

Current branch: `main` (Panchi backend) / `master` (Shunya OS frontend) **Remote:** `origin` → `git@github.com:shunya-os/Shunya-OS.git`
Uncommitted work: 8 modified files in Panchi backend, 14 modified files in Shunya OS frontend **Working product:** Yes, runs on Contabo VPS at `app.panchi.club`

Two working directories, NOT a monorepo: - `/root/panchi_backend/` — Flask backend (runs gunicorn on port 5000, active service) - `/root/shunya_os/` — Shunya OS unified application (templates, routes, models, packages)

The packages directory at `/root/shunya_os/packages/` contains 12 subdirectories: `cli`, `doctor`, `executor`, `foundation`, `governance`, `knowledge`, `planner`, `reasoning`, `runtime`, `shared`, `types`, `workflow`. **Most are empty** — only `foundation` has actual TypeScript code (`package.json`, `tsconfig`, `src/`). The rest are scaffolding directories with no source files.

Mismatch: The public GitHub `main` branch (`shunya-os/Shunya-OS`) has the old monolithic structure. The actual working product is split across two local directories with uncommitted changes on both. The `packages/` directory structure is documented and scaffolded but not implemented as TypeScript monorepo — the actual functionality lives in the Python `app/shunya/` directory.

2. COMPLETE APPLICATION MAP

Component	Directory	Framework	Runtime	Entry Point	Status
Panchi Backend	<code>/root/panchi_backend/</code>	Flask	Python 3.11	<code>wsgi.py</code> -> <code>app/init.py:create_app()</code>	ACTIVE (gunicorn systemd service)
Shunya OS Frontend	<code>/root/shunya_os/</code>	Flask + HTMX + Alpine.js	Python 3.11	<code>wsgi.py</code> -> <code>app/init.py:create_app()</code>	PARTIALLY ACTIVE (routes not registered in running service)
Shunya Pipeline	<code>/root/panchi_backend/app/shunya/</code>	Python	Python 3.11	<code>interface.py:ShunyaInterface</code>	ACTIVE (wired through <code>/shunya/*</code> API endpoints)
AI Assistant	<code>/root/panchi_backend/app/voice.py</code> , <code>companion.py</code> , <code>coach.py</code>	Python	Python 3.11	Inline in routes	ACTIVE (rule-based, not LLM)
WhatsApp Webhook	<code>/root/panchi_backend/app/whatsapp_webhook.py</code>	Flask	Python	Routes at <code>/whatsapp/*</code>	ACTIVE
Telegram Webhook	<code>/root/panchi_backend/app/routes.py</code>	Flask	Python	Routes at <code>/telegram/*</code>	ACTIVE
Worker (Celery)	<code>/root/panchi_backend/app/celery_worker.py</code>	Celery	Python	Config only	CONCEPTUAL (not running)
Database	PostgreSQL 16 + pgvector	SQLAlchemy	localhost:5432	<code>app/models.py</code>	ACTIVE (22 tables)
Cache	Redis (attempted)	In-memory fallback	localhost:6379	<code>app/cache.py</code>	DEGRADED (Redis not running, in-memory fallback)
Frontend Templates	<code>/root/shunya_os/templates/</code> (60+ HTML files)	Jinja2	Flask	Jinja2 ChoiceLoader	ACTIVE
Static Assets	<code>/root/shunya_os/static/</code>	CSS + JS	N/A	PWA manifest + <code>sw.js</code>	ACTIVE
Client Portal	<code>/root/panchi_backend/app/client_portal.py</code>	Flask	Python	Blueprint <code>client_bp</code>	ACTIVE

Missing / Not Running: - Celery worker — configured but not started - Redis — configured but not running - SMTP — configured in EmailAdapter but not connected - Scheduled jobs — no cron system

3. FRONTEND ARCHITECTURE

Framework: Flask served Jinja2 templates + HTMX 2.0 + Alpine.js 3.14 + Tailwind CSS CDN **No router** — server-side routing via Flask blueprints. Navigation is full-page loads (HTMX partial swaps for list/table updates).

Route map (Panchi backend): - `/` -> Dashboard (currently `dashboard.html`) - `/leads`, `/leads/new`, `/leads/<id>` -> Lead CRUD - `/payments`, `/invoices`, `/reports` -> Financial - `/settings`, `/tasks`, `/documents` -> Admin - `/whatsapp/webhook`, `/telegram/webhook` -> Channel webhooks - `/shunya/process`, `/shunya/knowledge`, `/shunya/summary` -> API

Layouts: Three base templates: - `base.html` (Shunya OS) — top nav + bottom nav + Bird AI widget + AI command bar - `base_universal.html` (Panchi) — sidebar + top bar + AI command bar - `auth_base.html` (Shunya OS) — standalone centered card for

login/signup

Navigation model: Currently **page-oriented** (top nav, bottom nav, or sidebar). Full page loads on navigation. HTMX is used only for partial content swaps, not for SPA-like navigation.

Responsive: Mobile-first via Tailwind breakpoints. Bottom nav bar on mobile, top nav or sidebar on desktop. PWA manifest with "display": "standalone" and "orientation": "portrait-primary".

State management: None. No client-side state store. Each page load fetches data from server. Alpine.js used for local UI state (toggles, dropdowns).

Design system: CSS variable-based theme engine with 4 themes (modern, playful, elegant, brandverse). shunya.css (50KB) has Tailwind fallbacks + theme variables. Data-theme attribute on .

Auth/Session: Flask session cookies. TeamMember model with password hashing. Login via /login/password (JSON API). OTP login partially implemented.

Specific Answers:

A. Page-oriented, not application-shell-oriented. Every navigation triggers a full page load. HTMX is used for content updates within pages, not for shell transitions.

B. Every navigation causes full page navigation -> layout remount -> context loss -> state reset. Partial HTMX swaps avoid full page reload for list/table updates only.

C. Can it evolve into a persistent single-surface application shell? Yes, but requires significant work: - Move the base.html template into a persistent shell with HTMX morphing/swapping - Currently hx-boost="true" on

enables partial page updates, but the shell (nav, sidebar) is reloaded on every navigation - The PWA manifest and service worker are already in place

D. Preserve: Design system (CSS variables, theme system), PWA setup, bottom nav mobile pattern, component structure within templates.

E. Incompatible with persistent app shell: Every page template is a full Jinja2 {% extends "base.html" %} — meaning the shell is re-rendered on every request. HTMX's hx-select="#main-content" and hx-target="#main-content" partially mitigate this, but the server still renders the full shell on every request.

4. MOBILE ARCHITECTURE

Viewport: Breakpoints: sm: 640px, md: 768px, lg: 1024px (Tailwind defaults) **Device detection:** data-device attribute (mobile/tablet/desktop) via JS on window resize **PWA:** Manifest with "display": "standalone", "orientation": "portrait-primary", service worker with cache-first strategy for shell assets **Mobile navigation:** Bottom nav bar with 5 items (Home, Records, AI, Money, Settings) + floating Bird AI button **Safe areas:** env(safe-area-inset-bottom,0px) for bottom nav padding **Virtual keyboard:** Not handled

Current mobile issues: - Sidebar on base_universal.html gets stuck open on mobile - Some 50KB response truncation issue - Content flow gaps (being fixed)

Portrait-first recommendations: - PWA orientation lock (portrait-primary) is already in manifest — enforceable in PWA mode only - In browser, responsive layout must handle both orientations - Application layout policy: bottom nav should always be visible, main content scrollable, no fixed-position overlays

5. BACKEND ARCHITECTURE

API architecture: Flask blueprints (main, api, auth_bp, client_bp). JSON responses for API, HTML for page routes.

Controllers/routes: Single routes.py (1529 lines) — monolithic. All CRUD logic, payment processing, webhooks, and API endpoints in one file.

Services: - services.py — inquiry text parsing, summary generation, Telegram token management - companion.py — greeting engine, banter, mood detection, suggestions (rule-based) - coach.py — coaching insights based on customer context - voice.py — voice command processing, keyword routing - creative.py — AI content generation (Instagram, Facebook, etc.) - notifications.py — notification management - celebrations.py — achievement celebration system - calendar_service.py — calendar event management

Shunya Pipeline (Domain Layer): - knowledge.py — knowledge base management, destination search, supplier lookup - reasoning.py — customer inquiry analysis, group type inference, strategy suggestion - planner.py — itinerary generation, proposal text generation - workflow.py — pipeline orchestration, lead creation - governance.py — validation, policy enforcement, approval - executor.py — multi-channel message delivery (WhatsApp, Telegram, Email) - interface.py — unified entry point for all channels - observer_learning.py — observation recording, learning signal generation - knowledge_store.py — immutable fact store with

versioning - data_import.py — CSV/Excel data import

Database access: SQLAlchemy ORM with PostgreSQL. 22 tables.

Auth: TeamMember model with password hashing, session-based auth, OTP login (partial), team management.

Authorization: UserRole enum (ADMIN, MANAGER, AGENT, VIEWER). login_required and admin_required decorators.

Request path for a new lead inquiry: 1. WhatsApp webhook -> routes.py:whatsapp_webhook() -> whatsapp_webhook.py:handle_whatsapp_incoming() 2. Parse text -> services.py:parse_inquiry_text() 3. Create lead -> models.py:Lead() 4. Or via Shunya Interface -> interface.py:ShunyaInterface.process_message() -> Knowledge -> Reasoning -> Planner -> Governance -> Executor -> Observer

6. DATA ARCHITECTURE

22 tables in PostgreSQL:

Table	Purpose	Domain
leads	Customer inquiries	TRAVEL
payments	Guest/supplier payments	FINANCE
suppliers	Supplier catalog	TRAVEL
invoices	Billing	FINANCE
itinerary_refs	Past itineraries	TRAVEL
task_lists	Task grouping	WORK
tasks	Individual tasks	WORK
team_members	Users	AUTH
tenants	Organizations	TENANCY
tenant_themes	Per-tenant theming	UI
notifications	In-app notifications	NOTIFICATIONS
activity_logs	Audit trail	AUDIT
documents	File uploads	STORAGE
knowledge_facts	Immutable fact store	KNOWLEDGE
observations	Learning observations	LEARNING
learning_entries	Learning signals	LEARNING
creative_assets	Generated content	CREATIVE
media_files	Media uploads	STORAGE
dynamic_fields	Custom fields	SCHEMA
dynamic_field_values	Custom field values	SCHEMA
module_blueprints	Module definitions	MODULES
feature_requests	User feedback	FEEDBACK

Key findings: - No unified Person model — customers are strings on Lead, team members are TeamMember, contacts exist only in Shunya OS Person model (not connected to running service) - No Relationship model in the running Panchi backend - Booking is a status string, not a first-class entity - The same human is represented differently across modules - Knowledge is versioned — KnowledgeFact has versioning, history, domain/category classification - Observations are persisted — Observation and LearningEntry tables exist

7. AI ASSISTANT — EXACT EXECUTION FLOW

Current AI Assistant is NOT an LLM — it's a rule-based command router.

User Message ("new lead for Sharma family, Bali, 2 adults") -> VoiceProcessor.process() /voice/process endpoint -> Keyword matching (no LLM, no API call) -> _handle_leads() -> parse_inquiry_text() -> create Lead -> Returns JSON response

Key determinations:

- RAG-only? NO — it's not even RAG. It's keyword-routing.
- Retrieve internal company knowledge? PARTIALLY
- Search live internet? YES — web_search.py has Serper API and DuckDuckGo support
- Has tools? CONCEPTUAL — internal method calls, not tool-calling LLM patterns
- Multi-step tool use? NO

- Conversational entity context? NO
- Act on company data? YES
- Create plans? YES — in-memory only
- Execute governed actions? PARTIAL
- Compare internal and external evidence? NO
- Track provenance? PARTIAL
- Track freshness? NO
- Distinguish fact from inference? NO
- Persist useful learning? PARTIAL — no feedback loop

8. PIPELINE INTEGRATION

Actual execution chain in process_message(): 1. KnowledgeLayer (via parse_inquiry_text -> no actual knowledge lookup) 2. ReasoningLayer.analyze_inquiry() -> uses knowledge.search_destination() 3. ReasoningLayer.suggest_approach() -> uses knowledge.get_past_itineraries() 4. PlannerLayer.create_itinerary() -> uses reasoning profile 5. PlannerLayer.generate_proposal_text() 6. GovernanceLayer.validate_plan() -> checks policies 7. ExecutorLayer.send() -> sends via WhatsApp/Telegram 8. ObserverLayer.observe() -> records observation

Intended SHUNYA loop mapped:

Stage	Status
Knowledge	IMPLEMENTED
-> Context	PARTIAL
-> Understanding	PARTIAL
-> Reasoning	IMPLEMENTED
-> Decision	PARTIAL
-> Plan	IMPLEMENTED
-> Workflow	PARTIAL
-> Execution	IMPLEMENTED
-> Observation	IMPLEMENTED
-> Learning	PARTIAL
-> Better Knowledge	MISSING

9. HUMAN AND RELATIONSHIP INTELLIGENCE READINESS

Codebase search results: - relationship: 0 in shunya/ pipeline. MISSING - memory: 0 in shunya/. MISSING - preference: 0 in shunya/. MISSING - sentiment: 0 in shunya/. MISSING - mood: 0 in shunya/. MISSING - provenance: 0 in shunya/. MISSING - freshness: 0 in shunya/. MISSING - confidence: 36 refs in governance.py, reasoning.py, planner.py - context: 27 refs - source: 43 refs in web_search.py, knowledge_store.py - evidence: 26 refs in governance.py, reasoning.py - persona: 2 refs in companion.py

Recommendation: These should be new domains/packages within the existing shunya/ architecture. The ShunyaInterface pipeline is the correct integration point — add a ContextLayer before ReasoningLayer.

10. PRIVACY AND MEMORY BOUNDARIES

Current state: No privacy/sensitivity classification exists. No consent tracking. No retention policy. No purpose limitation.

What exists: - ActivityLog tracks all actions with user id - Observation and LearningEntry persist data with no TTL - KnowledgeFact has no sensitivity classification - TeamMember has role-based access (ADMIN, MANAGER, AGENT, VIEWER) - No data deletion API exists - No audit trail for data access

Missing boundaries: - Session context vs long-term memory: NOT DISTINGUISHED - Organization exposure: NOT CONTROLLED - Commercial action authorization: LAYER exists but is minimal - Data deletion: NOT IMPLEMENTED - Health/political information: NOT HANDLED

11. WORLD INTELLIGENCE READINESS

What exists: - web_search.py — Serper API + DuckDuckGo search (both implemented) - web_search_available() — checks if search is configured - knowledge_store.py — immutable fact store (can serve as internal company data) - knowledge.py — knowledge base from markdown file

What's missing: - No news APIs - No RSS ingestion - No scheduled intelligence jobs - No source freshness evaluation - No conflict detection between internal and external data - No company-vs-external evidence comparison - No source normalization

12. FINAL ARCHITECTURE JUDGEMENT

1. What SHUNYA actually is today

A Flask-based travel CRM with a rule-based AI assistant, WhatsApp integration, and a multi-vertical ontology system. It has a working lead pipeline, payment tracking, invoice generation, and a two-database setup (PostgreSQL + pgvector). The frontend is a Jinja2 template system with HTMX partial updates, PWA capabilities, and a theme engine.

2. What is genuinely implemented

- Lead management (CRUD, status pipeline, activity logging)
- Payment tracking (guest/supplier, checkout, receipts)
- Invoice generation (with PDF)
- WhatsApp/Twilio webhook integration
- Multi-channel message executor (WhatsApp, Telegram, Email adapters)
- Rule-based AI assistant (keyword routing, companion, coaching)
- Knowledge store (immutable, versioned facts)
- Observation/learning system (persists observations, generates learning signals)
- Ontology system (multi-vertical dashboard configurator)
- Theme engine (4 CSS variable themes, PWA manifest, service worker)
- 60+ frontend templates across 12 verticals

3. What is mostly architectural foundation

- packages/ TypeScript monorepo — empty scaffolding
- Shunya Pipeline (Knowledge->Reasoning->Planner->Governance->Executor->Observer) — wired in code but not production-hardened
- Governance layer — minimal rules, no real policy enforcement
- AI Assistant — rule-based, not LLM-powered
- Celery worker — configured but not running
- Redis — configured but not running

4. What is currently disconnected

- Shunya OS vs Panchi backend — two separate Flask apps
- Person model — exists in Shunya OS but not in running database
- Relationship model — exists in Shunya OS but not connected
- packages/ TypeScript code — nothing imports it
- Learning feedback loop — observations never fed back into knowledge/reasoning

5. Can the frontend evolve into a persistent single-surface app shell?

Yes, but requires server-side rendering of only the content region (not the full shell), HTMX hx-boost refinement, and client-side state management.

6. Can the AI Assistant evolve into the proposed Context Fusion / Relationship Intelligence model?

Architecturally yes, but requires LLM integration, conversation memory, person model unification, relationship model connection, and tool-calling architecture.

7. Five largest architectural risks

1. Two-codebase problem — Panchi and Shunya OS are separate directories with separate git histories
2. No LLM integration — The AI Assistant is keyword routing
3. Monolithic routes.py — 1529 lines, all business logic in one file
4. No persistent plans — PlannerLayer creates in-memory plans
5. No feedback loop — Observations never consumed by Knowledge or Reasoning

8. Five strongest architectural assets

1. Shunya Pipeline architecture — Knowledge->Reasoning->Planner->Governance->Executor->Observer
2. Immutable Knowledge Store — KnowledgeFact with versioning
3. Multi-channel adapter pattern — WhatsApp, Telegram, Email

- Multi-vertical ontology system
- PWA + theme engine

9. What must NOT be rewritten

- Knowledge store (knowledge_store.py)
- Multi-channel executor (executor.py)
- Ontology system (ontology.py)
- Theme engine (CSS variables, shunya.css)
- PWA manifest + service worker
- Lead pipeline

10. What requires a new first-class architectural concept

- Person/Unified Identity model
- Relationship model
- Conversation/Memory system
- LLM Integration layer
- Plan persistence
- Learning feedback loop
- World intelligence system

CURRENT ARCHITECTURE MAP

```
app.panchi.club (Contabo VPS)
|
|   +--- Gunicorn (port 5000) wsgi:app
|   |
|   |   +--- Flask Application
|   |   |
|   |   |   +--- Auth Middleware (login_required, admin_required)
|   |   |   +--- Context Processor (nav_modules, user, notifications)
|   |   |   +--- Error Handler (JSON/HTML 404, 500)
|   |   |   |
|   |   |   +--- Blueprints:
|   |   |   |   auth_bp -> /login, /logout, /team
|   |   |   |   main    -> /, /leads, /payments, /invoices, /reports,
|   |   |   |           /settings, /tasks, /documents, /telegram,
|   |   |   |           /whatsapp, /voice, /calendar
|   |   |   |   api     -> /shunya/process, /shunya/knowledge,
|   |   |   |           /shunya/summary, /payment/create,
|   |   |   |           /notifications, /celebrations
|   |   |   |   client_bp -> /client/* (client portal)
|   |   |   |
|   |   |   +--- Shunya Pipeline:
|   |   |   |   KnowledgeLayer -> ReasoningLayer -> PlannerLayer
|   |   |   |   -> GovernanceLayer -> ExecutorLayer -> ObserverLayer
|   |   |   |   -> LearningLayer (No feedback loop)
|   |   |   |
|   |   |   +--- AI Assistant:
|   |   |   |   VoiceProcessor (keyword routing)
|   |   |   |   CompanionEngine (greeting/banter)
|   |   |   |   CoachEngine (rule-based insights)
|   |   |   |   NO LLM integration
|   |   |   |
|   |   |   +--- Database: PostgreSQL 16 + pgvector (22 tables)
|   |   |   +--- Redis: NOT RUNNING (in-memory fallback)
|   |   |   +--- Celery: NOT RUNNING (configured)
|   |   |
|   |   +--- Frontend Templates (Jinja2 ChoiceLoader):
|   |   |   1. /root/shunya_os/templates/ (60+ templates)
|   |   |   2. /root/panchi_backend/templates/ (30+ templates)
|   |   |   HTMX 2.0 + Alpine.js 3.14 + Tailwind CDN + PWA
|   |   |
|   |   +--- Two working directories:
|   |       /root/panchi_backend/ - Flask backend (active)
```

RECOMMENDED EVOLUTION SEQUENCE

1. **Consolidate the two codebases** — Register Shunya OS blueprints in the Panchi Flask app OR merge the two directories. This is the highest-impact risk.
2. **Integrate LLM** — Replace rule-based AI Assistant with a tool-calling LLM pattern. The ShunyaInterface pipeline is the right integration point.
3. **Person model unification** — Create a unified Person entity that replaces Lead.customer_name (string), TeamMember, ClientUser, and Shunya OS Person.
4. **Persist plans** — PlannerLayer output should be saved to a plans table, making plans actionable and trackable.
5. **Close the learning loop** — Feed ObserverLayer observations and LearningLayer signals back into KnowledgeStore and ReasoningLayer.
6. **World intelligence** — Add scheduled retrieval, freshness evaluation, and source conflict detection to the KnowledgeLayer.
7. **Privacy boundaries** — Add sensitivity classification, consent tracking, retention policies, and data deletion API.
8. **Application shell** — Refactor base template into a persistent shell with HTMX morphing. Client-side state with Alpine.js stores.